

FitBuddy - AI Fitness Plan Generator using Gemini Models

Project Documentation

Team Leader	N.Kalaimathi
Team Members	Member 1: A.Arunthathi Member 2: S.Anushiya Member 3: E.Rohini
Institution	H.H. The Rajah's College, Pudukkottai
Date	October 2026

Project Description

FitBuddy is a web-based application that uses AI to generate personalized workout plans and nutrition tips based on a user's fitness goals, such as weight loss, muscle gain, or general wellness. Built with FastAPI and integrated with Google's Gemini AI models, FitBuddy simplifies the process of creating structured and effective fitness routines.

The application takes in user information (name, age, weight, goal and preferred workout intensity) and generates a customized 7-day fitness plan. Users can also submit feedback to update and refine their plan using AI. A complementary nutrition or recovery tip is provided based on the selected goal.

With a clean user interface and database support via SQLite, FitBuddy ensures each user's data and generated plan are stored and accessible. It offers both API endpoints and an interactive HTML interface for ease of use.

Scenarios

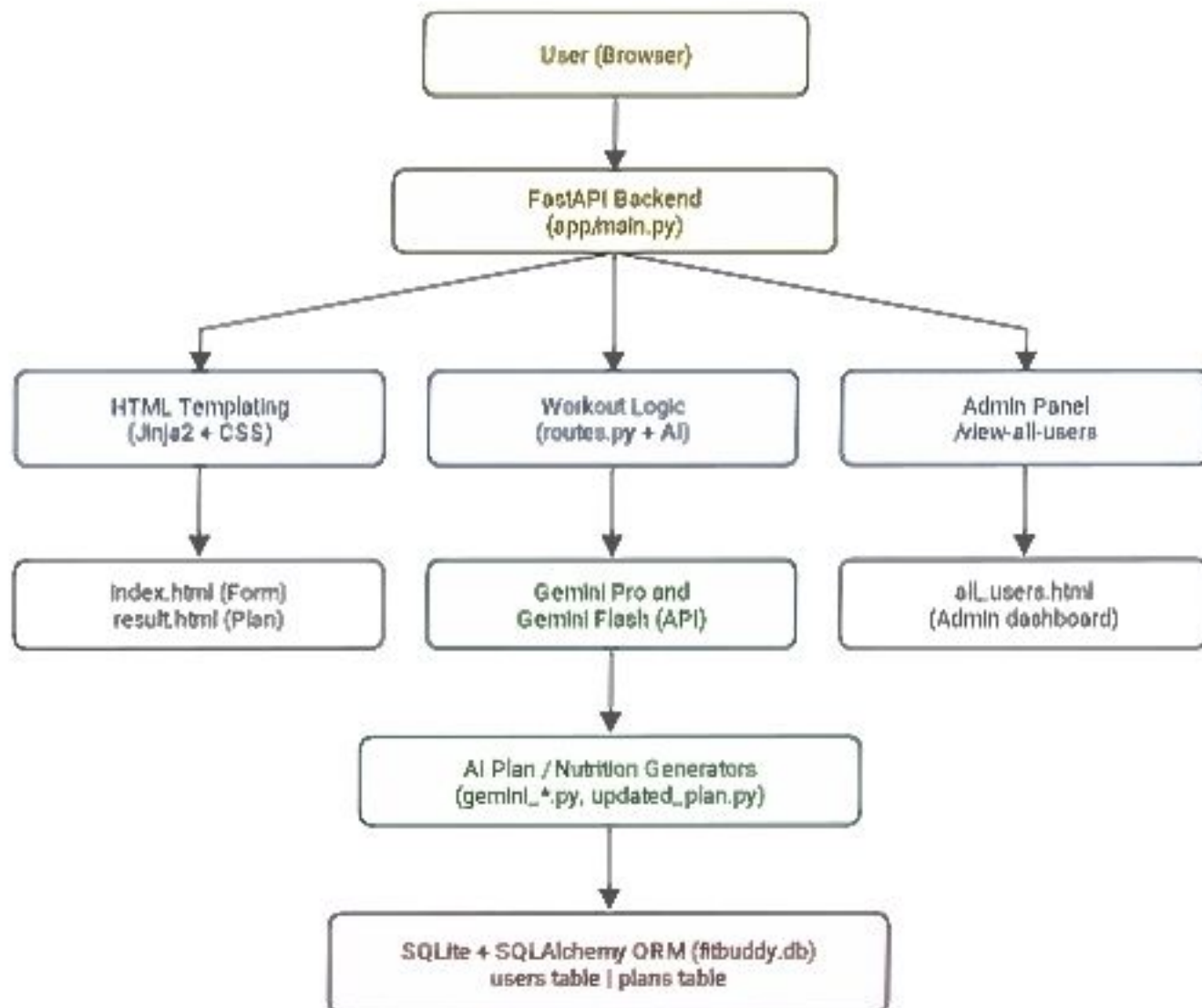
Scenario 1: A user enters their name, age, weight, fitness goal (for example "weight loss" or "muscle gain") and workout intensity (high, medium or low) to receive a personalized 7-day workout plan. The system generates a day-by-day schedule tailored to the user's objective.

Scenario 2: A user submits feedback on the existing plan, such as "more focus on cardio" or "include more rest days". The system regenerates an updated plan based on that feedback, keeping users engaged as their needs change.

Scenario 3: A user receives a short nutrition or recovery tip based on the selected goal (for example, "Include protein in your post-workout meal" for muscle gain), complementing the workout plan.

Scenario 4: An admin or coach opens a dashboard to view all registered users with their original and updated plans, making it easy to monitor several clients at once.

Technical Architecture



Pre-requisites

- FastAPI framework knowledge - fastapi.tiangolo.com
- Gemini API familiarity - ai.google.dev (Google Generative AI documentation)
- HTML, CSS and Jinja2 template skills
- Python programming proficiency
- Version control with Git
- SQLAlchemy and SQLite basics
- Environment setup (pip, virtualenv or conda)
- Uvicorn ASGI server

Project Workflow

- **Milestone 1:** Model selection and architecture (1.1 model research, 1.2 architecture, 1.3 environment setup)
- **Milestone 2:** Core functionalities development (2.1 core functions, 2.2 FastAPI backend)
- **Milestone 3:** routes.py development (3.1 main application logic)
- **Milestone 4:** Frontend development (4.1 user interface, 4.2 Jinja2 templates)

- **Milestone 5:** Deployment (5.1 local deployment, 5.2 testing and verification)

Milestone 1: Model Selection and Architecture

This milestone selects the generative AI models from Google's Gemini family and defines how the application is structured.

Activity 1.1: Research and Select the Appropriate Generative AI Model

FitBuddy's AI model needs to generate structured 7-day workout plans, provide concise nutrition tips, update plans from user feedback, and respond quickly enough for a web application.

Several Hugging Face models (T5, BART, DistilBERT, Bloom) were considered first. They showed promise in basic text generation but needed self-hosting or a local GPU, plus fine-tuning or complex prompting to produce structured plans.

Final selection: Google Gemini models, used through an API.

- **Gemini Pro** - structured 7-day workout generation and feedback-based plan updates
- **Gemini Flash** - fast, lightweight nutrition and recovery tips

Reasons for choosing Gemini: it is API-based and easy to integrate with FastAPI, naturally follows structured formatting (days, sections), gives accurate output with minimal prompt tuning, and is fast enough for feedback-based revisions. Model names are read from the .env file, so they can be changed when Google retires or releases models.

Activity 1.2: Define the Architecture of the Application

- **Frontend:** HTML + Jinja2 templates (index.html, result.html, all_users.html)
- **Backend:** FastAPI for route handling and logic
- **AI layer:** Google Gemini APIs (Pro and Flash)
- **Database:** SQLite with SQLAlchemy for persistence

The backend receives form data, calls the Gemini models, stores user information and plans, and renders dynamic templates.

Activity 1.3: Set Up the Development Environment

Create and activate a virtual environment

```
python -m venv fitbuddy-env
fitbuddy-env\Scripts\activate      # Windows
source fitbuddy-env/bin/activate   # macOS / Linux
```

requirements.txt

```
fastapi>=0.110
uvicorn
jinja2
sqlalchemy
python-multipart
google-generativeai
python-dotenv
```

Install and run

```
pip install -r requirements.txt
uvicorn app.main:app --reload
# App: http://127.0.0.1:8000      API docs: http://127.0.0.1:8000/docs
```

Project structure

```
fitbuddy/
|-- requirements.txt
|-- .env                      # GOOGLE_API_KEY=your_key_here
```

```
|-- fitbuddy.db                # auto-generated SQLite database
|-- app/
|   |-- __init__.py
|   |-- main.py                # FastAPI entry point
|   |-- routes.py              # route handlers
|   |-- database.py            # SQLAlchemy models + DB functions
|   |-- schemas.py             # Pydantic models
|   |-- gemini_generator.py     # Gemini Pro - workout plan
|   |-- gemini_flash_generator.py # Gemini Flash - nutrition tip
|   |-- updated_plan.py        # feedback-based plan updater
|   |-- templates/
|       |-- index.html
|       |-- result.html
|       |-- all_users.html
|-- static/
|   |-- css/style.css
|   |-- images/gym-bg.jpg
```

Milestone 2: Core Functionalities Development

Activity 2.1: Develop the Core Functionalities

Environment file (.env)

```
GOOGLE_API_KEY=your_gemini_api_key_here
GEMINI_PRO_MODEL=gemini-2.5-pro
GEMINI_FLASH_MODEL=gemini-2.5-flash
```

Workout plan generation - app/gemini_generator.py

generate_workout_gemini() sends the goal and intensity to Gemini Pro, which returns a day-wise plan with warm-up, main workout and cooldown.

```
import os
import google.generativeai as genai
from dotenv import load_dotenv

load_dotenv()
genai.configure(api_key=os.getenv("GOOGLE_API_KEY"))
model = genai.GenerativeModel(os.getenv("GEMINI_PRO_MODEL", "gemini-2.5-pro"))

def generate_workout_gemini(user_input: dict) -> str:
    prompt = f"""
    You are a professional fitness trainer.

    Create a personalized, structured 7-day workout plan for someone with the
    goal of **{user_input['goal']}**, who prefers **{user_input['intensity']}**
    intensity workouts.

    Each day must include:
    - A warm-up (5-10 mins)
    - Main workout (targeted exercises, sets & reps)
    - Cooldown or recovery tip

    Format:
    Day 1:
    Warm-up: ...
    Main Workout: ...
    Cooldown: ...
    (Repeat for Day 2-7)
    """
    try:
        response = model.generate_content(prompt)
        return response.text
    except Exception as e:
        return f"Error: {e}"
```

Nutrition tip generation - app/gemini_flash_generator.py

```
import os
import google.generativeai as genai
from dotenv import load_dotenv

load_dotenv()
genai.configure(api_key=os.getenv("GOOGLE_API_KEY"))
flash_model = genai.GenerativeModel(os.getenv("GEMINI_FLASH_MODEL", "gemini-2.5-flash"))

def generate_nutrition_tip_with_flash(goal: str) -> str:
    """Generate a nutrition or recovery tip for the user's fitness goal."""
    prompt = (
        f"Give one clear, helpful nutrition or recovery tip for someone focused on '{goal}'. "
        "The tip should be practical, friendly, and easy to understand."
    )
    try:
        response = flash_model.generate_content(prompt)
        return response.text.strip()
    except Exception as e:
        return f"Error generating tip: {str(e)}"
```

Feedback-based plan updating - app/updated_plan.py


```

import os
import google.generativeai as genai
from dotenv import load_dotenv

load_dotenv()
genai.configure(api_key=os.getenv("GOOGLE_API_KEY"))
model = genai.GenerativeModel(os.getenv("GEMINI_PRO_MODEL", "gemini-2.5-pro"))

def update_workout_plan(original_plan: str, user_feedback: str) -> str:
    """Use Gemini Pro to update the workout plan based on user feedback."""
    prompt = f"""
    You are a professional fitness trainer assistant.

    Here's the original 7-day workout plan:
    {original_plan}

    User Feedback:
    "{user_feedback}"

    Based on the feedback, revise the relevant parts of the workout plan.
    Keep the format and rest of the plan unchanged if not needed.
    """
    try:
        response = model.generate_content(prompt)
        return response.text.strip()
    except Exception as e:
        return f"Error updating plan: {e}"

```

User and plan storage - app/database.py

All user information and AI-generated plans are stored in SQLite through SQLAlchemy. The original and updated plans live in separate columns so both versions are preserved.

```

from sqlalchemy import create_engine, Column, Integer, String, Float, Text, ForeignKey
from sqlalchemy.orm import declarative_base, sessionmaker

engine = create_engine("sqlite:///./fitbuddy.db", connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(bind=engine, autoflush=False, expire_on_commit=False)
Base = declarative_base()

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True) # user_id entered in the form
    name = Column(String, nullable=False)
    age = Column(Integer)
    weight = Column(Float)
    goal = Column(String)
    intensity = Column(String)

class WorkoutPlan(Base):
    __tablename__ = "plans"
    id = Column(Integer, primary_key=True, autoincrement=True)
    user_id = Column(Integer, ForeignKey("users.id"))
    original_plan = Column(Text)
    updated_plan = Column(Text, nullable=True)

def init_db():
    Base.metadata.create_all(bind=engine)

def save_user(user_id: int, name: str, age: int, weight: float, goal: str, intensity: str):
    db = SessionLocal()
    existing = db.query(User).filter_by(id=user_id).first()
    if existing: # update existing user info
        existing.name, existing.age, existing.weight = name, age, weight
        existing.goal, existing.intensity = goal, intensity
    else: # create a new user
        db.add(User(id=user_id, name=name, age=age, weight=weight,
                    goal=goal, intensity=intensity))
    db.commit()
    db.close()

```



```

def save_plan(user_id: int, plan: str):
    db = SessionLocal()
    workout = db.query(WorkoutPlan).filter_by(user_id=user_id).first()
    if workout:
        # regenerated plan replaces the old one
        workout.original_plan = plan
        workout.updated_plan = None
    else:
        db.add(WorkoutPlan(user_id=user_id, original_plan=plan))
    db.commit()
    db.close()

def update_plan(user_id: int, updated_text: str):
    db = SessionLocal()
    workout = db.query(WorkoutPlan).filter_by(user_id=user_id).first()
    if workout:
        workout.updated_plan = updated_text
        db.commit()
    db.close()

def get_original_plan(user_id: int):
    db = SessionLocal()
    plan = db.query(WorkoutPlan).filter(WorkoutPlan.user_id == user_id).first()
    db.close()
    return plan.original_plan if plan else None

def get_user(user_id: int):
    db = SessionLocal()
    user = db.query(User).filter(User.id == user_id).first()
    db.close()
    return user

def get_all_users():
    db = SessionLocal()
    users = db.query(User).all()
    db.close()
    return users

def get_all_plans():
    db = SessionLocal()
    plans = db.query(WorkoutPlan).all()
    db.close()
    return plans

def delete_user(user_id: int):
    db = SessionLocal()
    db.query(WorkoutPlan).filter_by(user_id=user_id).delete()
    db.query(User).filter_by(id=user_id).delete()
    db.commit()
    db.close()

```

Activity 2.2: Implement the FastAPI Backend

Routing logic is handled in `routes.py`. The HTML form in `index.html` collects username, user_id, age, weight, goal and intensity; FastAPI captures them with `Form(...)` parameters, and Pydantic schemas validate the structure.

app/schemas.py

```

from pydantic import BaseModel

class UserInput(BaseModel):
    username: str
    user_id: int
    age: int
    weight: float
    goal: str
    intensity: str

```

```
class FeedbackRequest(BaseModel):
    feedback: str

class WorkoutRequest(BaseModel):
    goal: str
    intensity: str
```

app/main.py

```
import os
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles

from app.database import init_db
from app.routes import router

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

app = FastAPI(title="FitBuddy - AI Fitness Plan Generator")
app.mount("/static", StaticFiles(directory=os.path.join(BASE_DIR, "static")), name="static")

init_db()
app.include_router(router)
```

Milestone 3: routes.py Development

routes.py is the core operational layer. It connects the templates, the Gemini modules and the SQLite database.

Activity 3.1: Writing the Main Application Logic in routes.py

- / - displays the user form (index.html)
- **/generate-workout** - validates input, calls Gemini Pro and Flash, saves the user and plan, and renders result.html
- **/submit-feedback** - retrieves the original plan, revises it with Gemini Pro, saves the update and renders result.html
- **/view-all-users** - admin dashboard with all users and both plan versions
- **/delete-user/{user_id}** - removes a user and their plans from the admin dashboard
- JSON API routes: /generate-workout/gemini, /nutrition-tip, /generate-plan, /update-plan/{user_id}

app/routes.py

```
import os
from fastapi import APIRouter, Request, Form, HTTPException
from fastapi.responses import HTMLResponse, RedirectResponse
from fastapi.templating import Jinja2Templates

from app.schemas import UserInput, FeedbackRequest, WorkoutRequest
from app.gemini_generator import generate_workout_gemini
from app.gemini_flash_generator import generate_nutrition_tip_with_flash
from app.updated_plan import update_workout_plan
from app.database import (save_user, save_plan, update_plan, get_original_plan, get_user,
                          get_all_users, get_all_plans, delete_user)

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
templates = Jinja2Templates(directory=os.path.join(BASE_DIR, "templates"))
router = APIRouter()

# ----- Web pages -----
@router.get("/", response_class=HTMLResponse)
def home(request: Request):
    return templates.TemplateResponse(request, "index.html")

@router.post("/generate-workout", response_class=HTMLResponse)
```



```

def generate_workout(request: Request,
                    username: str = Form(...), user_id: int = Form(...),
                    age: int = Form(...), weight: float = Form(...),
                    goal: str = Form(...), intensity: str = Form(...)):
    data = UserInput(username=username, user_id=user_id, age=age,
                    weight=weight, goal=goal, intensity=intensity)
    plan = generate_workout_gemini({"goal": data.goal, "intensity": data.intensity})
    tip = generate_nutrition_tip_with_flash(data.goal)

    save_user(data.user_id, data.username, data.age, data.weight, data.goal, data.intensity)
    save_plan(data.user_id, plan)

    return templates.TemplateResponse(request, "result.html", {
        "username": data.username, "user_id": data.user_id, "age": data.age,
        "weight": data.weight, "goal": data.goal, "intensity": data.intensity,
        "workout_plan": plan, "nutrition_tip": tip, "message": None})

@router.post("/submit-feedback", response_class=HTMLResponse)
def submit_feedback(request: Request, user_id: int = Form(...), feedback: str = Form(...)):
    user = get_user(user_id)
    original = get_original_plan(user_id)
    if not user or not original:
        raise HTTPException(status_code=404, detail="No plan found for this User ID.")

    updated = update_workout_plan(original, feedback)
    update_plan(user_id, updated)
    tip = generate_nutrition_tip_with_flash(user.goal)

    return templates.TemplateResponse(request, "result.html", {
        "username": user.name, "user_id": user.id, "age": user.age,
        "weight": user.weight, "goal": user.goal, "intensity": user.intensity,
        "workout_plan": updated, "nutrition_tip": tip,
        "message": "Your plan has been updated based on your feedback!"})

@router.get("/view-all-users", response_class=HTMLResponse)
def view_all_users(request: Request):
    plans = {pl.user_id: pl for pl in get_all_plans()}
    user_data = []
    for u in get_all_users():
        plan = plans.get(u.id)
        user_data.append({
            "id": u.id, "name": u.name, "age": u.age, "weight": u.weight,
            "goal": u.goal, "intensity": u.intensity,
            "original_plan": plan.original_plan if plan else "N/A",
            "updated_plan": plan.updated_plan if plan and plan.updated_plan else "Not updated"})
    return templates.TemplateResponse(request, "all_users.html", {"users": user_data})

@router.post("/delete-user/{user_id}")
def remove_user(user_id: int):
    delete_user(user_id)
    return RedirectResponse(url="/view-all-users", status_code=303)

# ----- JSON API -----
@router.post("/generate-workout/gemini")
async def generate_gemini_workout(request: WorkoutRequest):
    try:
        result = generate_workout_gemini({"goal": request.goal, "intensity": request.intensity})
        return {"model": "gemini-pro", "workout_plan": result}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/nutrition-tip")
def get_flash_tip(goal: str):
    return {"goal": goal, "nutrition_tip": generate_nutrition_tip_with_flash(goal)}

@router.post("/generate-plan")
def generate_plan(user_data: UserInput):
    try:
        save_user(user_data.user_id, user_data.username, user_data.age,
                user_data.weight, user_data.goal, user_data.intensity)
        plan = generate_workout_gemini({"goal": user_data.goal,

```

```
        "intensity": user_data.intensity))
    save_plan(user_data.user_id, plan)
    return {"message": "Workout plan generated and saved successfully!",
            "workout_plan": plan}
except Exception as e:
    raise HTTPException(status_code=500, detail=f"Something went wrong: {str(e)}")

@router.post("/update-plan/{user_id}", response_model=dict)
def update_user_plan(user_id: int, data: FeedbackRequest):
    original = get_original_plan(user_id)
    if not original:
        return {"error": "Original plan not found for this user."}
    updated = update_workout_plan(original, data.feedback)
    update_plan(user_id, updated)
    return {"updated_plan": updated}
```


Milestone 4: Frontend Development

This milestone builds the user-facing pages with HTML, CSS and Jinja2. Each template is rendered dynamically through FastAPI.

Activity 4.1: Designing and Developing the User Interface

- `index.html` is the entry point and captures `username`, `user_id`, `age`, `weight`, `goal` and `intensity`.
- A gym-themed background image, Google Fonts (Roboto) and bold typography give a clean, fitness-oriented look.
- Flexbox centres the content, media queries handle mobile screens, and buttons have hover effects.
- Input fields and result blocks use padding, shadows and rounded corners.
- Three Jinja2 templates are used: `index.html`, `result.html` and `all_users.html`.

Styles are shared through one stylesheet (`static/css/style.css`) so the three pages stay consistent.

`app/static/css/style.css`

```
* { box-sizing: border-box; }
body {
  margin: 0; min-height: 100vh; font-family: 'Roboto', Arial, sans-serif; color: #1f2937;
  background: url('/static/images/gym-bg.jpg') center/cover fixed no-repeat;
  display: flex; justify-content: center; align-items: flex-start; padding: 30px 12px;
}
.card {
  background: rgba(255, 255, 255, 0.93); width: 100%; max-width: 480px;
  padding: 24px; border-radius: 12px; box-shadow: 0 8px 24px rgba(0, 0, 0, 0.3);
}
.card.wide { max-width: 900px; }
.card.full { max-width: 1300px; }
h1, h2 { text-align: center; margin-top: 0; }
label { display: block; font-weight: bold; margin: 12px 0 4px; font-size: 14px; }
input, select, textarea {
  width: 100%; padding: 10px; border: 1px solid #d1d5db; border-radius: 6px; font-size: 14px;
}
button, .btn {
  display: block; width: 100%; margin-top: 16px; padding: 11px; border: 0; border-radius: 6px;
  background: #3b82f6; color: #fff; font-size: 15px; text-align: center;
  text-decoration: none; cursor: pointer; transition: background 0.2s;
}
button:hover, .btn:hover { background: #2563eb; }
button.danger { background: #dc2626; margin-top: 0; padding: 6px; font-size: 13px; }
pre {
  white-space: pre-wrap; word-wrap: break-word; background: #f1f1f5;
  padding: 14px; border-radius: 8px; font-size: 13px; max-height: 480px; overflow-y: auto;
}
.success { color: #15803d; font-size: 18px; text-align: center; font-weight: bold; }
.info p { margin: 4px 0; }
table { width: 100%; border-collapse: collapse; background: #fff; font-size: 13px; }
th { background: #1e88e5; color: #fff; padding: 8px; text-align: left; }
td { padding: 8px; border-bottom: 1px solid #e5e7eb; vertical-align: top; }
td pre { max-height: 260px; font-size: 11px; }
@media (max-width: 600px) { .card { padding: 16px; } table { font-size: 11px; } }
```

Activity 4.2: Creating Dynamic Templates with FastAPI's Jinja2

Jinja2Templates is created in `routes.py`, and each route renders its page with `TemplateResponse` and a context dictionary. Form elements use `name="..."` attributes that match the backend `Form(...)` parameters.

`app/templates/index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>FitBuddy - AI Workout Generator</title>
  <link href="https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=swap" rel="stylesheet">
```

```

<link rel="stylesheet" href="/static/css/style.css">
</head>
<body>
  <div class="card">
    <h2>FitBuddy - AI Workout Generator</h2>
    <form action="/generate-workout" method="post">
      <label for="username">Name:</label>
      <input type="text" id="username" name="username" required>

      <label for="user_id">User ID:</label>
      <input type="number" id="user_id" name="user_id" required>

      <label for="age">Age:</label>
      <input type="number" id="age" name="age" min="10" max="100" required>

      <label for="weight">Weight (kg):</label>
      <input type="number" step="0.1" id="weight" name="weight" required>

      <label for="goal">Fitness Goal:</label>
      <input type="text" id="goal" name="goal" placeholder="e.g., weight loss, flexibility" required>

      <label for="intensity">Workout Intensity:</label>
      <select id="intensity" name="intensity">
        <option value="low">Low</option>
        <option value="medium">Medium</option>
        <option value="high">High</option>
      </select>

      <button type="submit">Generate Plan</button>
    </form>
    <a class="btn" href="/view-all-users">Admin: View All Users</a>
  </div>
</body>
</html>

```

app/templates/result.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Your Workout Plan</title>
  <link href="https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=swap" rel="stylesheet">
  <link rel="stylesheet" href="/static/css/style.css">
</head>
<body>
  <div class="card wide">
    <h1>Your Personalized Workout Plan</h1>
    {% if message %}<p class="success">{{ message }}</p>{% endif %}

    <h2>User Information</h2>
    <div class="info">
      <p><b>Name:</b> {{ username }}</p>
      <p><b>User ID:</b> {{ user_id }}</p>
      <p><b>Age:</b> {{ age }}</p>
      <p><b>Weight:</b> {{ weight }} kg</p>
      <p><b>Goal:</b> {{ goal }}</p>
      <p><b>Intensity:</b> {{ intensity }}</p>
    </div>

    <h2>Workout Plan</h2>
    <pre>{{ workout_plan }}</pre>

    <h2>Nutrition Tip</h2>
    <p>{{ nutrition_tip }}</p>

    <h2>Share Your Feedback</h2>
    <form action="/submit-feedback" method="post">
      <label for="user_id">Your Unique User ID:</label>
      <input type="number" name="user_id" id="user_id" value="{{ user_id }}"
        placeholder="Enter the same User ID used to generate your plan" required>
      <label for="feedback">Your Feedback:</label>
      <textarea name="feedback" id="feedback" rows="3"
        placeholder="Let us know how we can improve your plan..." required></textarea>
      <button type="submit">Submit Feedback</button>
    </form>

```

```

    <a class="btn" href="/">Back to Home</a>
  </div>
</body>
</html>

```

app/templates/all_users.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>FitBuddy - All Users</title>
  <link rel="stylesheet" href="/static/css/style.css">
</head>
<body>
  <div class="card full">
    <h1>FitBuddy - All Users & Workout Plans</h1>
    <table>
      <tr>
        <th>User ID</th><th>Name</th><th>Age</th><th>Weight (kg)</th><th>Goal</th>
        <th>Intensity</th><th>Original Plan</th><th>Updated Plan</th><th>Action</th>
      </tr>
      {% for user in users %}
      <tr>
        <td>{{ user.id }}</td>
        <td>{{ user.name }}</td>
        <td>{{ user.age }}</td>
        <td>{{ user.weight }}</td>
        <td>{{ user.goal }}</td>
        <td>{{ user.intensity }}</td>
        <td><pre>{{ user.original_plan }}</pre></td>
        <td><pre>{{ user.updated_plan }}</pre></td>
        <td>
          <form action="/delete-user/{{ user.id }}" method="post"
                onsubmit="return confirm('Delete this user?');">
            <button class="danger" type="submit">Delete</button>
          </form>
        </td>
      </tr>
      {% else %}
      <tr><td colspan="9">No users registered yet.</td></tr>
      {% endfor %}
    </table>
    <a class="btn" href="/">Back to Home</a>
  </div>
</body>
</html>

```


Milestone 5: Deployment

The application is deployed locally using FastAPI and Uvicorn. This milestone confirms that all dependencies are installed and the app runs correctly on a local server.

Activity 5.1: Preparing the Application for Local Deployment

- Create a virtual environment to isolate the project's dependencies.
- Install everything from requirements.txt (FastAPI, Uvicorn, SQLAlchemy, Google Generative AI SDK, python-dotenv).
- Store the Gemini API key in a .env file instead of hard-coding it, and never commit that file to Git.

```
python -m venv venv
source venv/bin/activate      # On Windows: venv\Scripts\activate
pip install -r requirements.txt

# .env
GOOGLE_API_KEY=your_gemini_api_key_here
```

Activity 5.2: Testing and Verifying Local Deployment

Start the server from the project root (the folder that contains the **app** folder):

```
uvicorn app.main:app --reload
```

Open **http://127.0.0.1:8000** for the application and **http://127.0.0.1:8000/docs** for the Interactive API documentation.

Home page

The homepage shows a centred form with Name, User ID, Age, Weight (kg), Fitness Goal and Workout Intensity (Low, Medium, High). The Generate Plan button triggers the AI backend.

Personalized workout plan page

After submission, the result page shows a summary of the user's details followed by the 7-day plan, with a warm-up, main workout and cooldown for each day.

Nutrition tip

A short, practical tip generated by Gemini Flash is shown beneath the plan to support the user's goal.

Feedback page

The user enters their User ID and feedback. Gemini Pro revises the plan, the update is saved, and a confirmation message appears.

View all users page

The admin page lists every registered user with their original and updated plans, and lets the admin delete a user.

Conclusion

FitBuddy is a user-centric web application that combines AI with fitness planning. Using Google's Gemini Pro and Flash models, it generates personalized 7-day workout plans and goal-based nutrition or recovery tips from a user's age, weight, goal and preferred intensity.

The backend is built with FastAPI, data is stored with SQLAlchemy and SQLite, and the interface is rendered with Jinja2. The feedback feature lets users refine their plan with AI, and the admin view gives coaches a clear picture of every user's original and updated plans.

Overall, FitBuddy is a scalable, adaptable solution that evolves with the user's progress and preferences.

Submitted by: **N.Kalaimathi**

H.H. The Rajah's College, Pudukkottai